

Bridge between Domain Specific Languages (DSL) and Domain Specific Architectures (DSA): Survey of DSL compilers and corresponding hardware accelerators

Aman Kalbiat

School of Engineering and Digital Sciences, department of Computer Science, 3rd year undergraduate

Nazarbayev University

Astana, Kazakhstan

aman.kalbiat@nu.edu.kz

Abstract—There were many scientist like Landauer [1], Bérut [2], Bennet [3] who investigated the ultimate limitations of computation. Based on some of their works and many of other's, Lloyd [4] come up with the upperbound of 5.4258×10^{50} operations per second for the ultimate laptop of mass 1 kilogram and volume of 1 liter. According to Intel Corporation [5], there is a processor which have a single core with 4.8×10^9 operations per second upper limit. One can notice that there is still 40 orders of magnitude difference. Since Moore's law stopped working, academia and industry should look into DSAs and DSLs in order to make their computations more efficient ([6], [7]). This paper proposes the survey of DSLs and DSAs according to their field of application and highlights some general algorithms and mathematical models used to develop such systems. In addition to that, it will look into various hardware solutions for different workflow and DSL mechanisms for optimization from different compilers. At the end the general rule for DSL/DSA codesign is proposed.

Index Terms—Heterogenous Computing, DSL, DSA, HPC

I. INTRODUCTION

Since industry started producing general purpose programmable computers, scientists were interested in the ultimate limitations of computations ([4], [1], [2], [3]). On the way to this conquest, Landauer [1] introduced the notion of irreversible logical operations and proposed the theoretical links between computation and entropy. His works were debated in academia, but Bennet [3] suggested clarification of the principles of Landauer and refuted for many of its critiques. In addition, Bennet discussed the Maxwell's Demon and why it does not defy 2nd Law of thermodynamics. In short, Maxwell's Demon is the system that stabilizes the entropy of the system using its knowledge about the system. But, the computation executed to make decisions about the system will introduce entropy. Then, Bérut et al [2] conducted an experiment showing that Landauer's theory worked. While Landauer [1], Bérut et al [2], and Bennet [3] were developing theory for calculation of ultimate limits, Von Neumann [8] proposed more practical theory on how to construct the working machine using components with some margin of error. He introduced the notion of majority organs in order to apply redundancy in

these faulty components and vote out the signals with large error. Afterwards, Lloyd [4] based on works of Landauer, Von Neumann and many others, come up with the estimation for the ultimate possible laptop with 1 kilogram of mass and 1 liter of volume. According to his estimates, the ultimate laptop would have performed 5.4258×10^{50} operations per second energy-wise. As an example, Intel Corporation's [5] processor specification shows that its single core performs up to 4.8×10^9 operations per second. One can notice that there is still approximately 40 orders of magnitude until the ultimate limit. Of course, these estimates are very rough and does not account for nuances like instructions per cycle, parallelism, floating point operation metric. However, it still shows that there is a plenty room for improvement. In the 20th century, this improvement was mainly based on miniaturization of the circuit, since it allowed to place more logical circuits on the same area of die ([9], [10]). In 1965, Moore [9] predicted that the number of transistors on the same unit of area will double approximately every 18 months. Then, in 1975 [10], he noticed that increase factor in density was 32 times for the time period from 1959 to 1975. Meaning, in average, doubling of logic circuits for the same area happened approximately every 2.2 years. But, according to both Horowitz [6], Hennessy & Patterson [7] this mechanism of miniaturization is hitting its physical limits. While Horowitz [6] looked into the power wall and cooling problems of such mechanism, Hennessy & Patterson [7] stated about the end of Moore's Law. According to Hennessy & Patterson [7] there is a 15-fold gap between Moore's prediction and actual density increase for the period of time from 2000 and 2018. Both [6] and [7] stressed the importance of the specialization of hardware for increasing efficiency. In addition to that, Hennessy & Patterson [7] mentioned the possible benefits of domain specific languages (DSL) in effective programming of targeted hardware of domain specific architectures (DSA). Generally, development of DSA and DSL are going to be the main directions in which people will gain speed and efficiency in computing tasks. Currently, there are many researchers investigating possible

DSAs for many applications ([11], [12], [13], [14], [15], [16]). Also, many scientists are looking into the possibilities of DSLs ([17], [18], [19], [20], [21], [14], [22], [23], [24]). This paper, proposes the survey of DSAs and corresponding DSLs according to its application (in-network computing, database management, bio-informatics, computer graphics) and points out general algorithms and applied mathematical models. Also it investigates hardware solutions to streamline some specific tasks and different DSL solutions. In section 2 there are given application specific overview. Then in section 3 there will be discussion of hardware solution for various applications. After tht section 4 will present different DSLs and their role in optimizing computations. Then in the last section there will be general suggestions for DSL/DSA codesign addressing all of the important tradeoffs investigated in this paper.

II. APPLICATION SPECIFIC OVERVIEW

As it was mentioned before, in the 1960s and 1970s Gordon Moor researched the trends in semiconductor manufacturing and inferred a pattern ([9], [10]). By this pattern density of transistors on the same area of chip die will double approximately every 18 months. However, it seems that this trend is hitting its physical limitations. Nowadays, Moore's law is not working as it was first projected in XX century. To reiterate, Hennessy and Patterson [7] for the period from 2000 up to 2018 the actual increase in density of the transistors is 15 times smaller than that of predictions given by Moore's Law. While Hennessy and Patterson [7] looked into the limitations of the CMOS technology for increasing the density of the transistors, Horowitz [6] investigated the power wall occurring for the cooling of dense stacked system on a chip. Both Hennessy and Patterson [7], Horowitz [6] suggested that next step of increase in computational power will come from specialization. While Horowitz [6] made an emphasis on domain specific hardware, Hennessy and Patterson [7] also mentioned domain specific languages (DSLs) as another method of increasing efficiency of computations. They stated that with programming DSAs with DSLs could potentially optimize the computing process by generating code which provides efficient mapping between logical operations and hardware resources. We present our collection of specialization solution on Table I. Here the proposed DSLs and DSAs are collected according to its field of application, computational operation that it focuses on, and target platforms.

III. HARDWARE SPECIALIZATION

One of the most famous example of hardware specialization is GPU. These are the speacial hardware that is specifically designed to perform tasks as matrix multiplication, dot product, and etc., which would help to speed up the graphics computations.

A. Parallelism vs. Locality

It is important to find a good trade off between Parallelism and Locality while designing architectures for heavy tasks. For example, according to Li et al [14], in the image processing

pipelines, kernels sometimes might require multiple stages of stencil operations. There must be good deal of locality and parallelism for efficient implementation. The authors mention that in their system that translates the existing Halide code to the FPGA netlist, their system uses HeteroCL as IR to get the optimal result in the search space. They stated that experiments demonstrated 4.15 times speed up compared to a system with 28 CPU cores while running on Xilinx FPGA. In addition to that, Fang et al. al [25] introduced an evaluation model which showed that the granularity of the system matters (cache line sizes) in hash-join operations. According to researchers, no-partitioning hash-joins often outperformed the radix partitioning hash joins. While no-partitioning hash-joins mostly rely on parallelism, radix partitioning hash joins tries to decrease the translation lookaside buffer (TLB) misses (optimized the locality). It seems that in radix partitioning hash joins, increase of radix bits would increase the overhead of build-up, which in term would negatively affect the performance, since the granularity of the system is fixed, and hash table could be be fit in the one block of the cache. All of the tests were performed on Intel Xeon E5-2670 v2 2.5GHz and IBM POWER8E 3.7GHz.

B. Memory bandwidth

Sometimes, bottleneck for the accelerating engines might be the memory bandwidth. If the device can provide high in device parallelism for computations, but lacks memory transfer speed, then this low memory bandwidth might be a bottleneck for efficient implementation of an algorithm. Fang et al [12], states that there might be multiple solutions for the memory bandwidth problems of FPGA accelerators. First of them is having a direct link between host and device so that they could work as co-processor. Secondly, utilizing High Bandwidth Memory in par with providing common memory space to the device and host. Another example for the bottleneck of the memory bandwidth could be the article by Zhou et al [11]. In this article, researchers proposed an accelerator for hierarchical index-based merge-join. Their architecture provided a special operation called range-check to enable the process of index traversal. Their cost analysis demonstrated that the data transfer cost is the main bottleneck. So they had to come up with the good trade-off between the parallelism and data-transfer. They did experimental studies based on TPC-H benchmark. They stated that the system gained 4.8 speed-up on the equi-join operations where the match rate was around 5 percent. Another solution for the problem of transfer bandwidth came from the area of in-network computing. Kianpisheh et al [13] stated an use-case of line-rate switches for ML/aggregation operation. According to the researches each switch will collect the result of ML/aggregation from its children switches, then perform the operation and forward it to the parental switch. Then all of the information will be collected in the analyzing server, and it will perform the final ML/aggregation operation. Due to the partial performing of the operations on the way of transferring the packet, the bandwidth of the network could be saved. In addition to that, the system proposed by Wu et al [16]

Paper	Application	Computation	Method	Platform
Zhou et al [11]	Database Management	Join operation	H	FPGA
Fang et al [12]	Database Management	Compressing/Decompressing, Queries offloading	H	FPGA
He et al [17]	Computer Graphics	Real-time shading	S	GPU
Bangaru et al [18]	Computer Graphics	Real-time shading	S	GPU
Li et al [19]	Machine Learning	Deep Learning	S	CPU, GPU, ASIC (TPU, Inference, NNP), DSP
Chen et al [20]	Machine Learning	Deep Learning	S	CPU, GPU, TPU
Zhao et al [21]	Machine Learning	Deep Learning	S	GPU
Kianpisheh and Taleb [13]	In-network computing	Caching, security, analytics, coordination	S/H	Programmable switch/router, Smart NIC, FPGA,
Li et al [14]	Image processing	Stencil operations	S/H	FPGA
Wu et al [15]	DNA sequencing	haplotype calling, variant calling, genotype calling	H	ASIC
Wu et al [16]	DNA sequencing	measuring nanopole signals	H	FPGA
Ragan-Kelley et al [22]	Image processing	Stencil operations	S	CPU, GPU
Sivaraman et al [23]	In-network computing	Packet processing	S/H	Line-rate switches (Banzai architecture)
Bosshart et al [24]	In-network computing	Packet processing	S	SDN
Fang et al [25]	Database Management	Hash join	H	CPU(Intel XEON, IBM POWER8E)

TABLE I

IN THE METHOD COLUMN, H MEANS HARDWARE (DSA), S MEANS SOFTWARE (DSL), S/H MEANS BOTH DSL AND DSA ARE INCLUDED

might serve as an another example for the application of CPU-FPGA communication solutions. In their paper, researchers looked into the process of analyzing the time-series signal from the nanopore sensors to identify base pairs (A-T, C-G). Their system implements the HMM algorithm that offloads the computation heavy part of the algorithm to the FPGA. They implemented the device-host communication via Reusable Integration Framework for FPGA Accelerators (RIFFA) on PCIe. They stated that, their system provides the speedup of 100 times and energy efficiency of one order of magnitude compared to CPU only solutions.

C. Expressiveness vs. Ease of implementation

Sometimes, it might be difficult to find a good trade off between expressiveness of the instructions executed on the hardware and architectural complication of the hardware. If the hardware is too simple, the expressiveness needed to implement algorithms might get harmed, even if simple architectures can provide much higher speed. As an example, Sivaraman et al [23], proposed an architecture with relatively not sophisticated features, but which can help to implement various packet processing algorithms using their expressive language called Domino. The code written in Domino can be compiled down to the Banzai architecture machine, which is specifically designed to process packets. Base structures of the architecture is match-action tables consisting of many atoms. Their tests showed that in Domino data-plane algorithms can be implemented with 2-3 times less lines of code over 11 different algorithms targeting Banzai machines. Furthermore, over specialization of hardware on some specific task might also harm the flexibility of the system. For example, Wu et al [15] proposed a system which can perform genotyping, variant calling, short-read mapping, and haplotype calling. These operations are used in Next-Generation Sequencing of

DNA. Researchers designed an architecture which includes dynamic programming engine, k-mer processing engine, and variant discovery engine. Experimental results demonstrated that it achieved 3-to-59 times higher throughput than other existing solutions. In addition to that, the chip achieves 935 times higher energy efficiency compared to Illumina DRAGEN FPGA system. However it has following shortcomings: does not support long reads from 3rd generation sequencing, fixed design parameters for alignment process (cannot be modified), default values are used for read-to-allele matrix derivation. Which means, very inflexible.

IV. DSL COMPILERS

Domain Specific Languages are specialized languages designed to freely express a specific task. Sometimes, they are created as an extension of some existing programming language targeting specific application.

A. Multiple hardware targets

It is important for modern DSLs to target various hardware. But it might introduce complications in mapping the logical operations to the hardware. Because each architecture (especially specialized) needs its own optimization loops during the lowering process. Li et al [19] addressed this in the scope of deep learning DSL. They surveyed various compilers as TVM, nGraph, TC, Glow, and XLA. Authors stated that for portability reasons, the compiler backend must compile to multiple IRs like Halide based, Polyhedral based, HLO, or other unique IRs to support various hardware specific optimizations. Of course, before that, there should be a comprehensive compiler front-end which would first translate the written program into efficient computational graph (like ONNX) before lowering the code. One of the DSL compiler for Deep learning that was stated above was TVM. It was presented by Chen et al

[20]. They stated that accelerators based on CPU, GPU, and TPU each have their own distinct memory architecture. So, this variance must be addressed when generating the optimized code. Experiments conducted by researchers demonstrated that TVM generated code which has speed-up of at least 1.2 times up to 3.8 times over existing hand-optimized libraries. Additionally, Bosshart et al [24] investigated the programmable packet processors and come up with P4 as a strawman proposal for OpenFlow protocol. The main premises of P4 are: hardware independence, reconcilability in the field, and protocol independence. The authors argue that approach of having a reconfigurable language and compilers that generates low level codes for targets will eventually result in better utilization of software defined networks.

B. Algorithm agnostic abstraction

In some cases, it might be better to hide some of the algorithm details into the compiler to develop more efficient systems. As an example Automatic Differentiation system proposed by Bangaru et al [18] can deal with differentiable objects automatically. The Slang D proposed by authors can identify the differentiable objects by introducing an extension for Slang language that has special differentiable types. Their system supports higher order differentiation and produces high performance derivative code. Experiments on the microbenchmark showed that the kernels generated by Slang D was much faster than PyTorch kernels and at the same speed as CUDA kernels. Meaning, this Automatic Differentiation system could allow gradient based techniques without harming the performance. While Bangaru et al [18] looked into the automatic differentiation for real-time shading systems, Zhao et al [21] come-up with their own system for deep learning that has convolution, pooling, and activation as a part of their grammar for constructing the AST. They proposed a language called DeepDSL which is based on Scala. It will compile it to Java code which can be executed in CPU+GPU systems for acceleration. Compiler will handle the mapping of such operations as convolution, pooling, and activation. Their system also enables differentiable programming as in the case of Slang D. As mentioned before, enabling the differentiated programming, helps to utilize gradient based methods. They made experiments on Linux server with a single Nvidia K40c GPU. Results of comparison with Caffe (another DSL): 77 percent faster in Overfeat, 69 percent faster in GoogLeNet, 88 percent faster in AlexNet. Moreover, Ragan-Kelley et al [22] proposed a DSL targeting CPUs and GPUs which would automatically define the optimal schedule for image processing computations. Usually, such systems would target a very specific hardware, and optimizations would be done by a specialist whose sole job is optimizing such pipelines. The speed-up during this scenario would be painstakingly gained by the people manually developing and testing such system. Halide, a DSL proposed by authors makes this process much easier. In order to program in Halide, developer must express the pipeline on terms of what to calculate rather than how to calculate. It is done by the simple declarative forms of the

Halide and a special schedule. Experiments demonstrated that Halide provides 5 times speed-up compared to hand-tuned C-kernels and CUDA implementations. Tests were executed on quad core Xeon W3520 x86 CPU, and NVIDIA Tesla C2070 GPU.

C. Ease of introduction

If the development of system requires a full-stack hardware-software development knowledge, then, it will decrease the domain of people who would be able to develop such systems. Fang et al [12] stated that in the development of accelerators for database management systems it was mostly considered that the developer should know both the higher abstraction of the queries both at the level of algorithms and hardware. However, HLS languages like Glacier could resolve this issue by acting as an efficient mapper of the needed queries into the FPGA net-list. Another good way of developing new language systems might be developing extensions for already existing languages, or at least creating languages that would not defy the common sense of that application. For example, He et al [17], proposed a system for real time shading systems which would be used as an extension for already existing C-like shading language HLSL. They identified modern programming language features that proprietary systems lacked and introduced them into the extension so that it would facilitate developer productivity and not harm the efficiency of hardware specific kernels that it will generate. The proposed system requires the developer to express the shader object both in shading language and host C language. Then Slang would use these two files and generate kernels which would accommodate for type of materials and type of light used in the scene. It was successfully tested in the Falcor system which demonstrated time improvements both on CPU and GPU. Refactored code demonstrated up to 50 percentage drop in time.

V. ETHICAL ISSUES

The main premise of this paper is to survey different approaches in the design of DSLs and DSAs. It is mostly applicable for increasing the efficiency of computations in terms of speed of calculations, memory usage, and energy consumption. It does not directly touch areas that might inflict harm to individuals or the society as whole.

VI. RESULT AND CONCLUSION

As the conclusion of the paper, list of suggestions for DSL/DSA codesign is presented:

- Preserve domain information deeper into the compiler. The more quickly a DSL lowers into generic IR, the harder it becomes to exploit domain structure at the hardware level.
- Design hardware-aware abstractions, not just pretty syntax. The language should expose scheduling, locality, data movement, and synchronization decisions where the backend needs them.
- Use multi-target IRs carefully. They improve portability, but should not diminish the performance gain among GPUs, FPGAs, and ASIC-like accelerators

The unresolved challenge is: identifying the general hardware architecture building method, that would contain all of the needed solution space of that application and allowing the construction of DSLs for it.

REFERENCES

- [1] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961.
- [2] A. Bérut, A. Arakelyan, A. Petrosyan, S. Ciliberto, R. Dillenschneider, and E. Lutz, "Experimental verification of landauer's principle linking information and thermodynamics," *Nature*, vol. 483, no. 7388, pp. 187–189, 2012.
- [3] C. H. Bennett, "Notes on landauer's principle, reversible computation, and maxwell's demon," *Studies in History and Philosophy of Modern Physics*, vol. 34, no. 3, pp. 501–510, 2003.
- [4] S. Lloyd, "Ultimate physical limits to computation," *Nature*, vol. 406, no. 6799, pp. 1047–1054, 2000.
- [5] Intel Corporation, "Intel core ultra 7 processor 258v specifications," 2024. Accessed: 2026-03-26.
- [6] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14, IEEE, 2014.
- [7] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.
- [8] J. von Neumann, "Probabilistic logics and the synthesis of reliable organisms from unreliable components," in *Automata Studies* (C. E. Shannon and J. McCarthy, eds.), vol. 34 of *Annals of Mathematics Studies*, pp. 43–98, Princeton University Press, 1956.
- [9] G. E. Moore, "Cramming more components onto integrated circuits," *Electronics*, vol. 38, no. 8, pp. 114–117, 1965. Reprinted in Proceedings of IEEE, 1998.
- [10] G. E. Moore, "Progress in digital integrated electronics," in *Technical Digest of the International Electron Devices Meeting (IEDM)*, pp. 11–13, IEEE, 1975.
- [11] Z. Zhou, C. Yu, S. Nutanong, Y. Cui, C. Fu, and C. J. Xue, "A hardware-accelerated solution for hierarchical index-based merge-join," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 1, pp. 91–104, 2019.
- [12] J. Fang, Y. T. B. Mulder, J. Hidders, J. Lee, and H. P. Hofstee, "In-memory database acceleration on fpgas: a survey," *The VLDB Journal*, vol. 29, no. 0, pp. 33–59, 2020.
- [13] S. Kianpisheh and T. Taleb, "A survey on in-network computing: Programmable data plane and technology specific applications," *IEEE COMMUNICATIONS SURVEYS TUTORIALS*, vol. 25, no. 1, pp. 701–761, 2023.
- [14] J. Li, Y. Chi, and J. Cong, "Heterohalide: From image processing dsl to efficient fpga acceleration," in *FPGA '20: Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2020.
- [15] Y.-C. Wu, Y.-L. Chen, C.-H. Yang, C.-H. Lee, W.-C. Chen, L.-Y. Lin, N.-S. Chang, C.-P. Lin, C.-S. Chen, J.-H. Hung, and C.-H. Yang, "A 28nm fully integrated end-to-end genome analysis accelerator for next-generation sequencing," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 19, no. 6, pp. 1105–1119, 2025.
- [16] Z. Wu, K. Hammad, E. Ghafar-Zadeh, and S. Magierowski, "Fpga-accelerated 3rd generation dna sequencing," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 14, no. 1, pp. 65–74, 2020.
- [17] Y. He, K. Fatahalian, and T. Foley, "Slang: language mechanisms for extensible real-time shading systems," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 141:1–141:13, 2018.
- [18] S. P. Bangaru, L. Wu, T. Li, J. Munkberg, G. L. Bernstein, and J. Ragan-Kelley, "Slang.d: Fast, modular and differentiable shader programming," *ACM Transactions on Graphics*, vol. 42, no. 6, pp. 264:1–264:28, 2023.
- [19] M. Li, Y. Liu, X. Liu, Q. Sun, X. You, H. Yang, Z. Luan, L. Gan, G. Yang, and D. Qian, "The deep learning compiler: A comprehensive survey," *IEEE Transactions On Parallel and Distributed Systems*, vol. 32, no. 3, pp. 708–727, 2021.
- [20] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowin, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, 2018.
- [21] T. Zhao and X. Huang, "Design and implementation of deepdsl: A dsl for deep learning," *Computer Languages, Systems Structures*, vol. 54, no. 0, pp. 39–70, 2018.
- [22] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *ACM SIGPLAN Notices*, vol. 48, no. 6, p. 519–530, 2013.
- [23] A. Sivaraman, A. Cheung, M. Budiu, C. Kim, M. Alizadeh, H. Balakrishnan, G. Varghese, N. McKeown, and S. Licking, "Packet transactions: High-level programming for line-rate switches," in *SIGCOMM '16: Proceedings of the 2016 ACM SIGCOMM Conference*, pp. 15–28, 2016.
- [24] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, p. 87–95, 2014.
- [25] J. Fang, J. Lee, P. Hofstee, and J. Hidders, "Analyzing in-memory hash joins: Granularity matters," in *Proceedings of the 8th International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures*, pp. 1–8, 2017.